



와인 품질 분류

미니 프로젝트(2)

YB 1조 김재경, 유재은, 이우림, 전예원

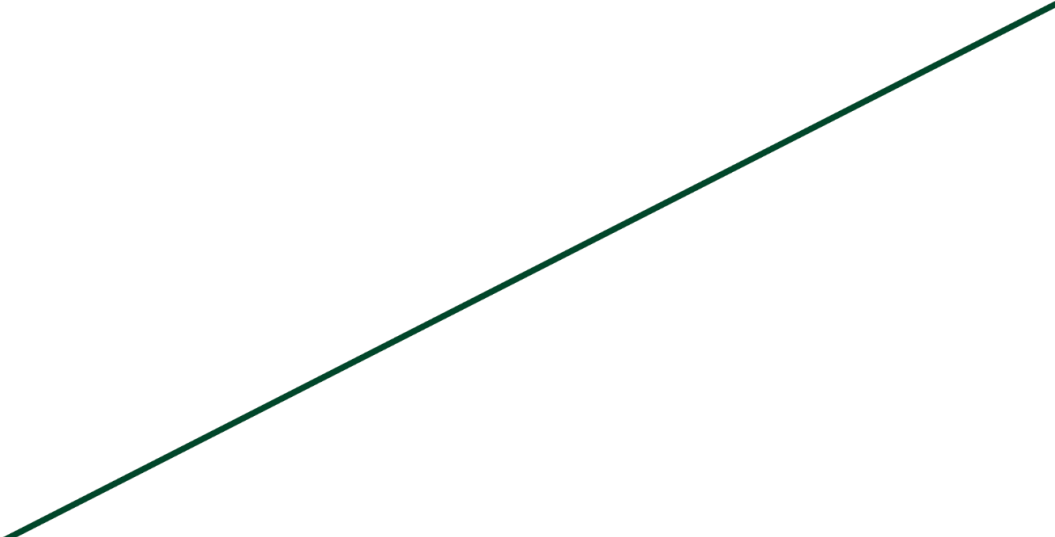
목차

1. 간단한 EDA

2. Method 1

3. Method 2

4. Method 3



프로젝트 소개 및 데이터 설명

Train 데이터: 총 13개의 변수(12개 독립변수 + 품질 변수)

```
# train 데이터의 개형을 살펴봅니다.  
# index를 제외하면 총 13개 변수를 가집니다.  
train.head()
```

Python

	index	quality	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	type
0	0	5	5.6	0.695	0.06	6.8	0.042	9.0	84.0	0.99432	3.44	0.44	10.2	white
1	1	5	8.8	0.610	0.14	2.4	0.067	10.0	42.0	0.99690	3.19	0.59	9.5	red
2	2	5	7.9	0.210	0.39	2.0	0.057	21.0	138.0	0.99176	3.05	0.52	10.9	white
3	3	6	7.0	0.210	0.31	6.0	0.046	29.0	108.0	0.99390	3.26	0.50	10.8	white
4	4	6	7.8	0.400	0.26	9.5	0.059	32.0	178.0	0.99550	3.04	0.43	10.9	white

Test 데이터: 12개의 독립변수만(품질 변수 제외)

```
# test 데이터의 개형을 살펴봅니다.  
# index를 제외하면 총 12개 변수를 가집니다.  
# train 중 quality 변수가 사라졌습니다.  
test.head()
```

Python

	index	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	type
0	0	9.0	0.31	0.48	6.6	0.043	11.0	73.0	0.99380	2.90	0.38	11.6	white
1	1	13.3	0.43	0.58	1.9	0.070	15.0	40.0	1.00040	3.06	0.49	9.0	red
2	2	6.5	0.28	0.27	5.2	0.040	44.0	179.0	0.99480	3.19	0.69	9.4	white
3	3	7.2	0.15	0.39	1.8	0.043	21.0	159.0	0.99480	3.52	0.47	10.0	white
4	4	6.8	0.26	0.26	2.0	0.019	23.5	72.0	0.99041	3.16	0.47	11.8	white

데이터 로딩 및 기본 구조

Pandas를 이용해 데이터 불러오기, 결측치 여부 확인

```
train = pd.read_csv("train.csv")
test = pd.read_csv("test.csv")
```

```
# train 데이터의 열 정보를 살펴봅니다.
# 결측치는 없습니다.
train.info()
```

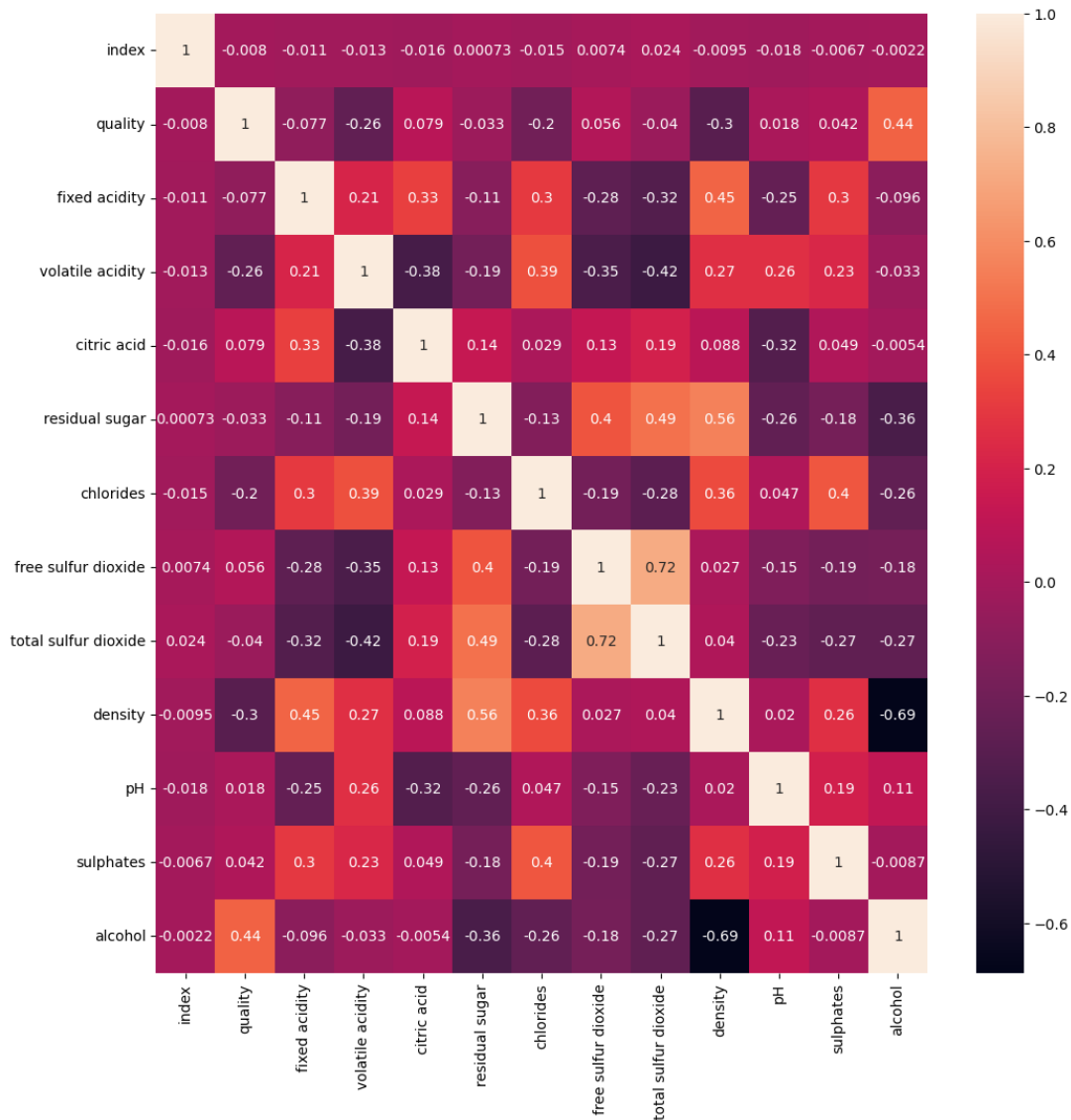
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5497 entries, 0 to 5496
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   index                 5497 non-null  int64
1   quality               5497 non-null  int64
2   fixed acidity         5497 non-null  float64
3   volatile acidity     5497 non-null  float64
4   citric acid           5497 non-null  float64
5   residual sugar        5497 non-null  float64
6   chlorides             5497 non-null  float64
7   free sulfur dioxide   5497 non-null  float64
8   total sulfur dioxide  5497 non-null  float64
9   density              5497 non-null  float64
10  pH                   5497 non-null  float64
11  sulphates            5497 non-null  float64
12  alcohol              5497 non-null  float64
13  type                 5497 non-null  object
dtypes: float64(11), int64(2), object(1)
memory usage: 601.4+ KB
```

상관관계 분석

Heatmap으로 변수 간 상관관계 확인하기

```
# train의 변수 간 상관관계를 살펴봅니다.  
plt.figure(figsize=(12,12))  
sns.heatmap(data = train.select_dtypes(include=[np.number]).corr(), annot=True)  
plt.show()
```

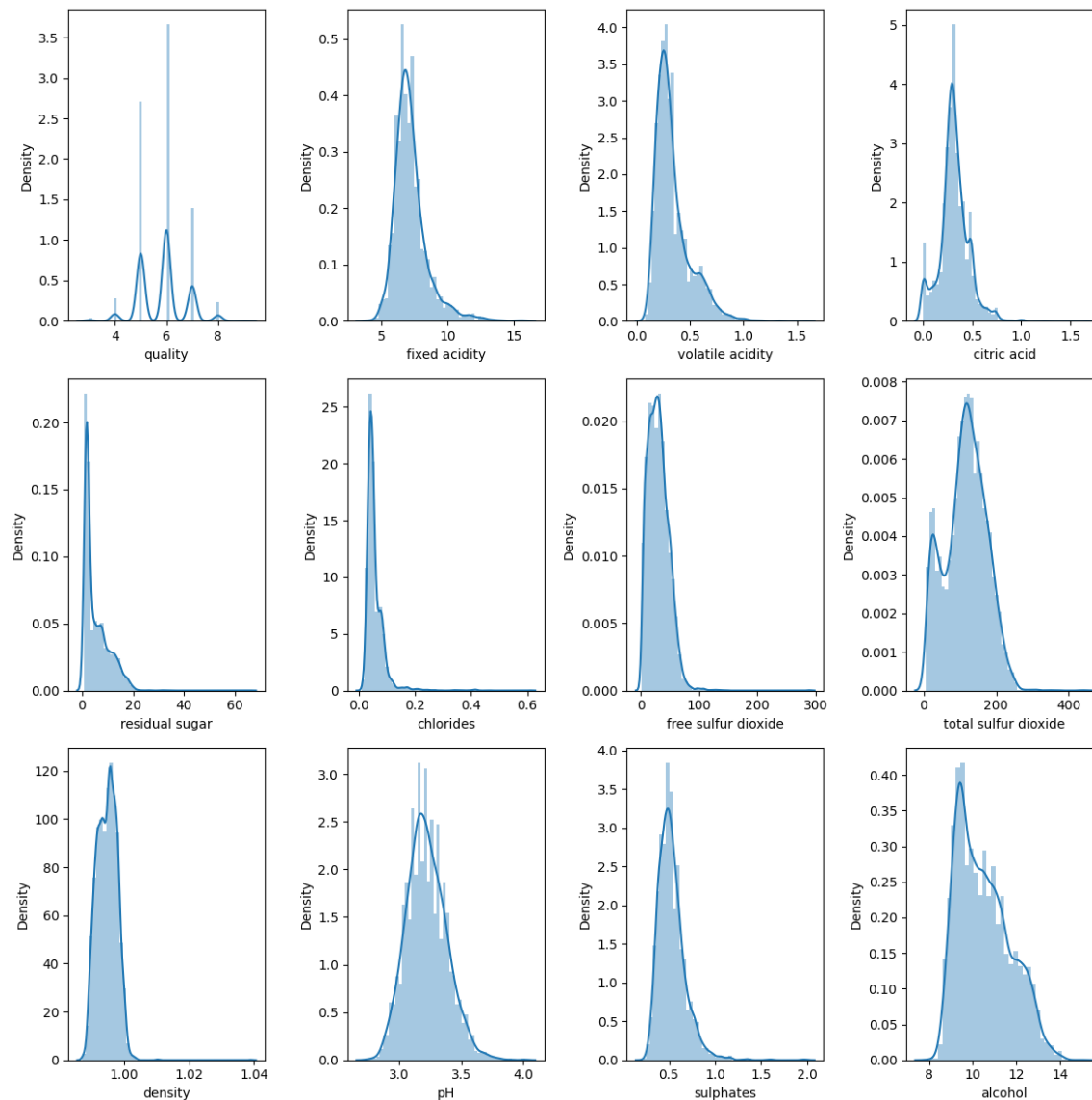
alcohol과 density, volatile acidity
-> quality와 상관성이 높음



변수별 분포 시각화

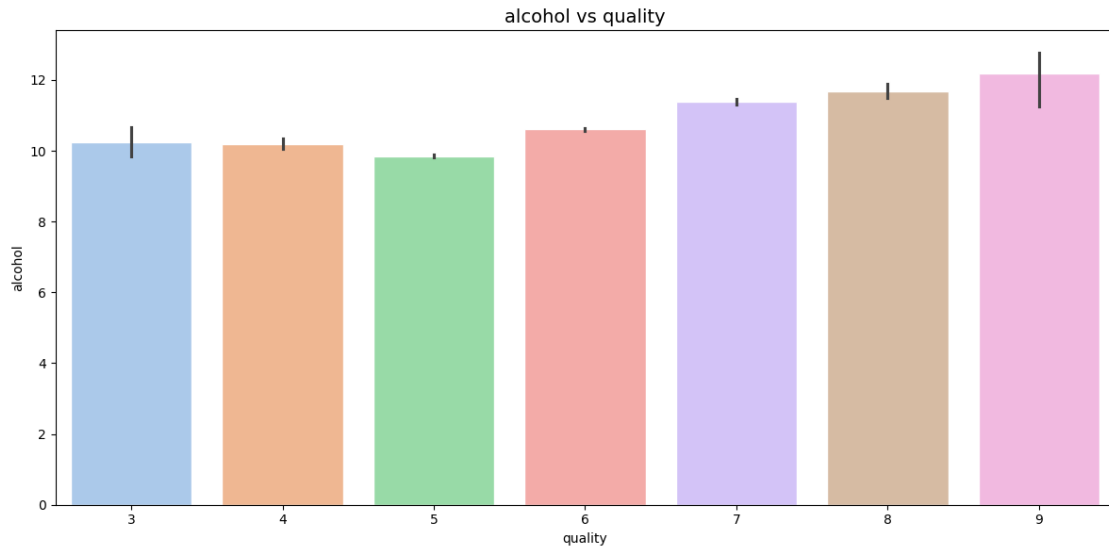
대부분 오른쪽으로 꼬리가 긴 형태

```
# train의 각 변수별 분포를 살펴봅니다.
plt.figure(figsize=(12,12))
for i in range(1,13):
    plt.subplot(3,4,i)
    sns.distplot(train.iloc[:,i])
plt.tight_layout()
plt.show()
```

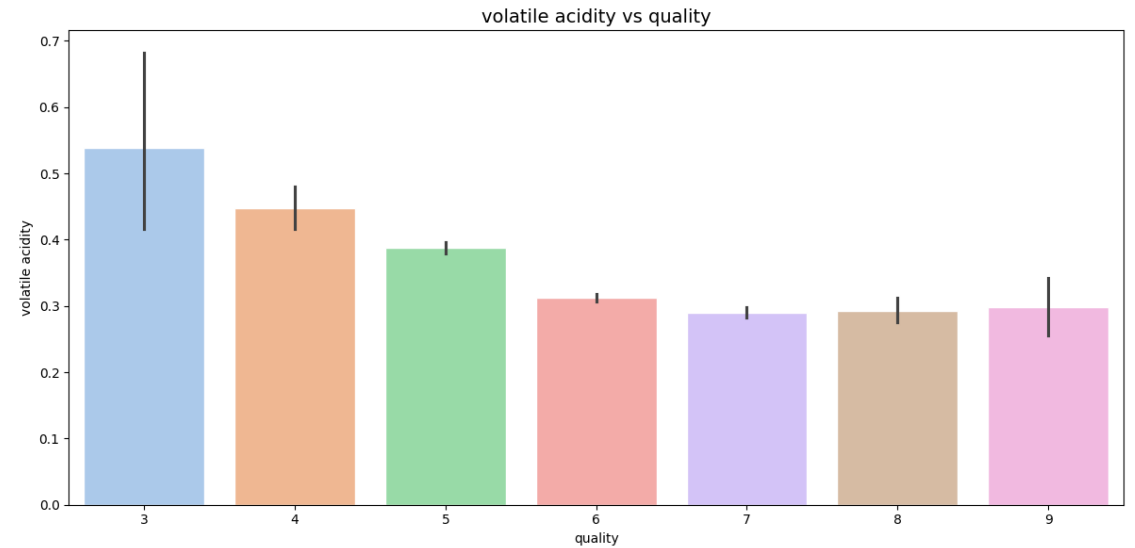


각 변수와 quality 변수 사이 분포 확인

alcohol vs quality



Volatile acidity vs quality



Method 1: 데이터 전처리

```
from sklearn.preprocessing import LabelEncoder

enc = LabelEncoder()
enc.fit(train['type'])
train['type'] = enc.transform(train['type'])
test['type'] = enc.transform(test['type'])
```

문자(범주형) 변수 type을 숫자 형태로 변환하기 위해
LabelEncoder 사용
red, white를 각각 0,1로 매핑
train과 test에 같은 규칙으로 변환 적용

```
from sklearn.model_selection import train_test_split

train_x = train.drop(['index', 'quality'], axis=1)
train_y = train['quality']
test_x = test.drop(['index'], axis=1)

y_encoded = train_y - train_y.min()
```

train_x: 모델이 학습할 입력 데이터(독립변수)
train_y: 예측할 대상인 품질 점수(종속변수)
index는 분석에 불필요하므로 제거
품질 점수가 3~9이므로 0~6으로 변환

Method 1: 데이터 전처리

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(train_x)
X_test_scaled = scaler.transform(test_x)
```

MinMaxScaler를 사용해 모든 피쳐 값을 0~1 사이로 변환
train 데이터로 스케일을 학습하고, 동일한 규칙을 test 데이터에 적용

Method 1: 모델링

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

rf = RandomForestClassifier(random_state=42)

param_grid_rf = {
    'n_estimators': [300, 500, 700],
    'max_depth': [5, 7, 9, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2', None]
}

grid_rf = GridSearchCV(rf, param_grid_rf, cv=3, scoring='f1_macro', n_jobs=-1)
grid_rf.fit(X_scaled, y_encoded)

print("RF Best Params:", grid_rf.best_params_)
```

랜덤 포레스트 분류 모델을 사용해서 가장 성능이 좋은 하이퍼파라미터 조합을 찾음

- 모델의 성능에 영향을 주는 주요 설정값들을 지정
- GridSearchCV로 최적 조합 탐색
- 탐색 결과 가장 성능이 좋은 파라미터 조합을 출력

RF Best Params: {'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 700}

Method 1: 모델링

```
from sklearn.ensemble import RandomForestClassifier  
  
rf = RandomForestClassifier(**RF_Best_Params)  
  
rf.fit(X_scaled, y_encoded)
```

GridSearchCV로 찾은 최적의 하이퍼파라미터를 사용해서 랜덤 포레스트 모델을
최종적으로 학습
전처리된 학습 데이터로 모델을 훈련

Method 1: 학습된 모델로 test 예측

```
y_pred = rf.predict(X_test_scaled)
```

학습이 완료된 랜덤 포레스트 모델로 테스트 데이터를 예측

```
y_pred += 3
```

예측 결과는 0~6 범위이므로 품질점수를 원래의 3~9 품질 점수 범위로 변환

Method 1: 제출 파일 생성 및 점수

```
submission = pd.read_csv('sample_submission.csv')
```

제출 형식 불러오기

```
submission['quality'] = y_pred
```

y_pred를 quality 열에 넣음

```
submission.to_csv('rf_minmax.csv', index=False)
```

CSV 파일로 저장

	index	quality
0	0	5
1	1	6
2	2	6
3	3	5
4	4	6
...	...	...
995	995	6
996	996	6
997	997	5
998	998	6
999	999	6

rf_minmax.csv

메모 추가

최종 점수: 0.702

2025-11-04
19:33:56

0.702



Method 2: 데이터 전처리

```
from sklearn.preprocessing import LabelEncoder

enc = LabelEncoder()
enc.fit(train['type'])
train['type'] = enc.transform(train['type'])
test['type'] = enc.transform(test['type'])
```

Type의 white와 red 두 종류를 각각 1,0으로 변환

```
train_x = train.drop(['index', 'quality'], axis = 1)
train_y = train['quality']
test_x = test.drop('index', axis = 1)
```

불필요한 변수 제거

Method 2: 데이터 전처리

1) MinMaxScaler

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# 'type' 컬럼을 제외한 연속형 변수 리스트
continuous_cols = [col for col in train_x.columns if col != 'type']

# 스케일러 정의 및 연속형 변수에만 Fit/Transform 적용
scaler = MinMaxScaler()
train_x_scaled_cont = scaler.fit_transform(train_x[continuous_cols])
test_x_scaled_cont = scaler.transform(test_x[continuous_cols])

# 데이터프레임으로 변환
train_x_scaled = pd.DataFrame(train_x_scaled_cont, columns=continuous_cols, index=train_x.index)
test_x_scaled = pd.DataFrame(test_x_scaled_cont, columns=continuous_cols, index=test_x.index)

# 'type' 컬럼 (Label Encoding이 완료된 0 또는 1)을 다시 추가
train_x_scaled['type'] = train_x['type'].astype('int64')
test_x_scaled['type'] = test_x['type'].astype('int64')

# 최종적으로 CatBoost에 사용할 데이터프레임 완성

print(train_x_scaled.head())
print(test_x_scaled.head())
```

train_x에만 fit을 해주고,
test_x에는 fit을 하지 않음

⇒ test 데이터는 훈련을 위한
데이터가 아닌 예측을 위한
데이터임

'0'과 '1'을 명확히 정수 타입으로
설정하여 모델이 이를 범주로
인식하도록 함

⇒ CatBoost는 범주형 데이터가
정수(integer) 타입일 때
예측 오류나 성능 저하 없이
잘 작동함

Method 2: 데이터 전처리

- train_x_scaled

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	#
0	0.148760	0.410000	0.036145	0.095092	0.054908	
1	0.413223	0.353333	0.084337	0.027607	0.096506	
2	0.338843	0.086667	0.234940	0.021472	0.079867	
3	0.264463	0.086667	0.186747	0.082822	0.061564	
4	0.330579	0.213333	0.156627	0.136503	0.083195	

	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	#
0	0.027778	0.179724	0.139001	0.551181	0.123596	
1	0.031250	0.082949	0.188741	0.354331	0.207865	
2	0.069444	0.304147	0.089647	0.244094	0.168539	
3	0.097222	0.235023	0.130904	0.409449	0.157303	
4	0.107639	0.396313	0.161751	0.236220	0.117978	

	alcohol	type
0	0.318841	1
1	0.217391	0
2	0.420290	1
3	0.405797	1
4	0.420290	1

- test_x_scaled

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	#
0	0.429752	0.153333	0.289157	0.092025	0.056572	
1	0.785124	0.233333	0.349398	0.019939	0.101498	
2	0.223140	0.133333	0.162651	0.070552	0.051581	
3	0.280992	0.046667	0.234940	0.018405	0.056572	
4	0.247934	0.120000	0.156627	0.021472	0.016639	

	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	#
0	0.034722	0.154378	0.128976	0.125984	0.089888	
1	0.048611	0.078341	0.256217	0.251969	0.151685	
2	0.149306	0.398618	0.148255	0.354331	0.264045	
3	0.069444	0.352535	0.148255	0.614173	0.140449	
4	0.078125	0.152074	0.063621	0.330709	0.140449	

	alcohol	type
0	0.521739	1
1	0.144928	0
2	0.202899	1
3	0.289855	1
4	0.550725	1

Method 2: 데이터 전처리

2) 타겟 레이블 변환

```
#최소 품질 수 계산
min_score = train['quality'].min()
print(f"최소 품질 점수: {min_score}")

# 모든 점수에서 최소 점수(3)를 빼서 0부터 시작하는 레이블로 변환: 3 -> 0, 4 -> 1, 5 -> 2 ...
train_y_to_use = train_y.astype('int') - min_score
print(f"최소값: {train_y_to_use.min()}, 최대값: {train_y_to_use.max()}")

# 예측해야 하는 품질 점수의 수는 3,4,5,6,7,8,9로 7개임 (레이블값: 0,1,2,3,4,5,6)
num_classes = len(train_y_to_use.unique())
print(num_classes)
```

```
최소 품질 점수: 3
최소값: 0, 최대값: 6
7
```

XGBoost와 CatBoost를 사용할 것이므로 'quality'를 0부터 시작하는 연속된 정수로 변경해줌

Method 2: 모델링

1) 라이브러리 설치 및 импорт

```
!pip install catboost
```

```
Requirement already satisfied: catboost in /usr/local/lib/python3.12/dist-packages (1.2.8)
Requirement already satisfied: graphviz in /usr/local/lib/python3.12/dist-packages (from catboost) (0.21)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from catboost) (3.10.0)
Requirement already satisfied: numpy<3.0,>=1.16.0 in /usr/local/lib/python3.12/dist-packages (from catboost) (2.0.2)
Requirement already satisfied: pandas>=0.24 in /usr/local/lib/python3.12/dist-packages (from catboost) (2.2.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from catboost) (1.16.3)
Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packages (from catboost) (5.24.1)
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages (from catboost) (1.17.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=0.24->catboost) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=0.24->catboost) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=0.24->catboost) (2025.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (25.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (3.2.5)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.12/dist-packages (from plotly->catboost) (8.5.0)
```

```
from sklearn.model_selection import GridSearchCV
from xgboost import XGBClassifier
from catboost import CatBoostClassifier
```

Method 2: 모델링

2) XGBoost

```
# XGBoost 튜닝
xgb_param_grid = {
    'n_estimators': [500],
    'max_depth': [6, 8, 10],
    'learning_rate': [0.03, 0.05],
    'reg_lambda': [0.1, 1]}

xgb_base = XGBClassifier(
    objective='multi:softprob'
    use_label_encoder=False,
    eval_metric='mlogloss',
    random_state=2024,
    n_jobs=-1)

# GridSearchCV 실행 및 best_xgb_params 저장
xgb_grid_search = GridSearchCV(estimator=xgb_base, param_grid=xgb_param_grid,
                               scoring='accuracy', cv=3, n_jobs=-1, verbose=1)
xgb_grid_search.fit(train_x_scaled, train_y_to_use)
best_xgb_params = xgb_grid_search.best_params_
print(f"XGBoost 최고 정확도: {xgb_grid_search.best_score_: .4f}")

Fitting 3 folds for each of 12 candidates, totalling 36 fits
XGBoost 최고 정확도: 0.6371
```

[하이퍼파라미터 탐색 범위 정의]

- 트리의 개수: 500개
- 트리의 최대 깊이: 6 OR 8 OR 10
- 학습률: 0.03 OR 0.05
- L2 정규화 계수: 0.1 OR 1

[기본 XGBoost 모델 설정]

- 모델의 학습 목표
: 다중 클래스 분류의 확률 예측

[하이퍼파라미터 튜닝]

- 모델 성능 평가 지표: 정확도
- GridSearchCV를 사용하여 모든 12개의 조합을 3겹 교차 검증으로 학습시켜 최적의 조합을 찾음

Method 2: 모델링

3) CatBoost

```
# CatBoost 튜닝
cat_param_grid = {
    'n_estimators': [500],
    'depth': [6, 8, 10],
    'learning_rate': [0.03, 0.05],
    'l2_leaf_reg': [1, 3]}

cat_base = CatBoostClassifier(
    loss_function='MultiClass',
    classes_count=num_classes,
    verbose=0,
    random_state=2024,
    thread_count=-1,
    cat_features=['type'])

# GridSearchCV 실행 및 best_cat_params 저장
cat_grid_search = GridSearchCV(estimator=cat_base, param_grid=cat_param_grid,
                               scoring='accuracy', cv=3, n_jobs=-1, verbose=1)
cat_grid_search.fit(train_x_scaled, train_y_to_use)
best_cat_params = cat_grid_search.best_params_
print(f"CatBoost 최고 정확도: {cat_grid_search.best_score_:.4f}")

Fitting 3 folds for each of 12 candidates, totalling 36 fits
CatBoost 최고 정확도: 0.6489
```

[하이퍼파라미터 탐색 범위 정의]

- 트리의 개수: 500개
- 트리의 최대 깊이: 6 OR 8 OR 10
- 학습률: 0.03 OR 0.05
- L2 정규화 계수: 1 OR 3

[기본 CatBoost 모델 설정]

- 모델의 학습 목표
: 다중 클래스 분류의 확률 예측
- 모델이 예측해야 할 총 클래스의 개수
: num_classes 변수 값

[하이퍼파라미터 튜닝]

- 모델 성능 평가 지표: 정확도
- GridSearchCV를 사용하여 모든 12개의 조합을 3겹 교차 검증으로 학습시켜 최적의 조합을 찾음

Method 2: 모델링

4) 하이퍼파라미터를 이용한 개별 분류기 재정의

```
best_cat = CatBoostClassifier(  
    **best_cat_params,  
    loss_function='MultiClass',  
    classes_count=num_classes,  
    random_state=2024,  
    verbose=0,  
    thread_count=-1,  
    cat_features=['type'])
```

```
best_xgb = XGBClassifier(  
    **best_xgb_params,  
    objective='multi:softprob',  
    use_label_encoder=False,  
    eval_metric='mlogloss',  
    random_state=2024,  
    n_jobs=-1)
```

- 
- Grid Search에서 찾은 하이퍼파라미터 (best_cat_params)를 적용
 - 'type' 컬럼이 범주형 특성임을 명시하여, CatBoost가 내부적으로 이를 효율적으로 처리하도록 함

- 
- Grid Search에서 찾은 하이퍼파라미터 (best_xgb_params)를 적용

Method 2: 모델링

5) 앙상블 모델 구축 및 학습

```
from sklearn.ensemble import VotingClassifier

# 최적화된 모델로 Voting Classifier 재구성
final_vote_model = VotingClassifier(
    estimators=[("cat", best_cat), ("xgb", best_xgb)],
    voting='soft',
    n_jobs=-1)

# 최종 앙상블 모델 학습
final_vote_model.fit(train_x_scaled, train_y_to_use)

# 4. 최종 예측 및 원래 점수로 복원 (제출 준비)
y_pred = final_vote_model.predict(test_x_scaled)

# 예측값을 원래의 품질 점수 (3, 4, 5, ...)로 역변환
final_submission_predictions = y_pred.astype(int) + 3
```

- best_cat과 best_xgb를 앙상블에 포함시킴
- 보팅 방식: Soft Voting

- 앙상블 모델을 train_x_scale과 train_y_to_use를 사용하여 학습시킴

- 학습이 완료된 앙상블 모델을 test_x_scaled에 적용하여 y_pred를 예측함
- 예측값은 변환된 레이블(0, 1, 2, ...) 형태이므로 역변환이 필수

Method 2: 제출 파일 생성 및 점수

```
y_pred = final_vote_model.predict(test_x_scaled)

submission = pd.read_csv('sample_submission.csv')

submission['quality'] = final_submission_predictions

# csv 파일로 저장합니다.
submission.to_csv('quality5.csv', index=False)
```

quality5 >

	A	B	
1	index	quality	
2	0	5	
3	1	6	
4	2	6	
5	3	5	
6	4	6	
7	5	6	
8	6	6	
9	7	6	
10	8	6	

Method 3 데이터 전처리

1. 라벨 인코딩
2. 불필요한 변수 제거(index)
3. MinMaxScailing

변수 명:
train_x_mms
train_y
test_mms

Method 3 모델링

최적 데이터 찾기

> 랜덤포레스트 단일 모델 기준



Method 3 데이터 전처리

최적 데이터 찾기

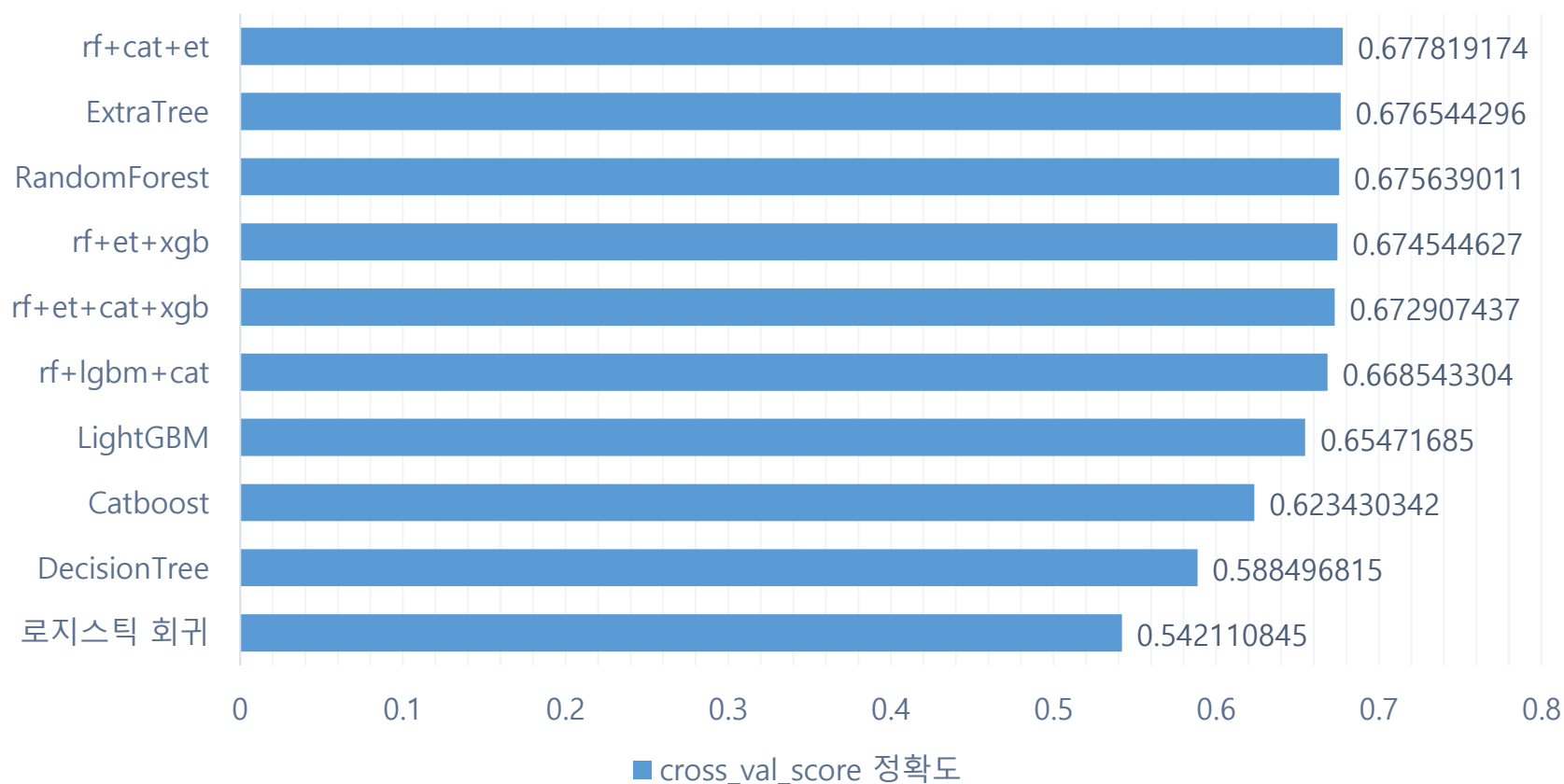
> 정규화만 한 경우가 가장 점수 높음

1345531	mp2_randomforest_over.csv 메모 추가	오버샘플링	2025-11-04 17:24:05	0.662
1345517	mp2_randomforest_iqx.csv 메모 추가	이상치제거	2025-11-04 17:06:27	0.634
1345266	mp2_randomforest_drop.csv 메모 추가	피쳐 드롭	2025-11-03 21:55:26	0.698
1345246	mp2_randomforest.csv randomforest+minmax edit	정규화	2025-11-03 20:49:45	0.699

Method 3 모델링

최적 모델 찾기

> 결정트리 제외 정규화 후 진행(합쳐진 모델은 보팅)



Method 3 모델링

최적 모델 찾기

> 정확도 가장 높은 rf+et+cat 모델 제출시 점수: 0.705

1345612 mp2_voting_rfetcat.csv

2025-11-04
23:24:29

0.705

```
# ✅ 각 모델 정의
rf = RandomForestClassifier(
    n_estimators=500, max_depth=20, random_state=42
)

lgbm = LGBMClassifier(
    n_estimators=500, learning_rate=0.05, max_depth=-1,
    random_state=42
)

cat = CatBoostClassifier(
    iterations=500, depth=8, learning_rate=0.05,
    loss_function='MultiClass', verbose=0, random_state=42
)

# ✅ 보팅 모델 (soft voting)
voting_clf = VotingClassifier(
    estimators=[('rf', rf), ('lgbm', lgbm), ('cat', cat)],
    voting='soft', # 각 모델의 확률 평균
    n_jobs=-1
)

# ✅ 교차검증으로 평가
scores = cross_val_score(voting_clf, train_x_mms, train_y, scoring='accuracy', cv=5)
```

Method 3 모델링

최적 모델 찾기

> 정확도 가장 높은 rf+et+cat 보팅 모델에서 개별 모델 그리드 서치

1346241	mp2_voting_rfetcat3.csv 메모 추가	개별 모델 그리드 <u>서치2</u>	2025-11-06 16:34:58	0.7
1346230	mp2_voting_rfetcat2.csv 메모 추가	개별 모델 그리드 <u>서치1</u>	2025-11-06 16:18:28	0.694

```
from sklearn.model_selection import GridSearchCV

cat = CatBoostClassifier(loss_function='MultiClass', verbose=0, random_state=42)

param_grid_cat = {
    'depth': [6, 8],
    'learning_rate': [0.03, 0.05],
    'iterations': [300, 500],
    'l2_leaf_reg': [3, 5, 7]
}

grid_cat = GridSearchCV(
    cat,
    param_grid=param_grid_cat,
    scoring='accuracy',
    cv=3,
    n_jobs=-1,
    verbose=2
)

grid_cat.fit(train_x_mms, train_y)

print("👉 Best Cat Params:", grid_cat.best_params_)
print("👉 Best Cat CV Accuracy:", grid_cat.best_score_)
```

Fitting 3 folds for each of 24 candidates, totalling 72 fits
👉 Best Cat Params: {'depth': 8, 'iterations': 500, 'l2_leaf_reg': 3, 'learning_rate': 0.05}
👉 Best Cat CV Accuracy: 0.6234303418406363

Method 3 모델링

최적 모델 찾기

> 정확도 가장 높은 rf+et+cat 보팅 모델에서 가중치 그리드 서치

```
voting = VotingClassifier(  
    estimators=[('rf', rf), ('et', et), ('cat', cat)],  
    voting='soft'  
)  
  
param_grid = {  
    'weights': [  
        [1, 1, 1],  
        [2, 1, 1],  
        [1, 2, 1],  
        [1, 1, 2],  
        [2, 2, 1],  
        [3, 2, 1]  
    ]  
}  
  
grid = GridSearchCV(voting, param_grid, cv=5, scoring='accuracy', n_jobs=-1, verbose=1)  
grid.fit(train_x_mms, train_y)  
  
print("Best Weights:", grid.best_params_['weights'])  
print("Best CV Accuracy:", grid.best_score_)
```

Fitting 5 folds for each of 6 candidates, totalling 30 fits
Best Weights: [1, 2, 1]
Best CV Accuracy: 0.6789107453056498

1346271	mp2_voting_rfetcat_weight121_3.csv 메모 추가	보팅 개별 모델 가중치[1,2,1]+ 하이퍼 파라미터 변경	2025-11-06 18:01:31	0.706
1346266	mp2_voting_rfetcat_weight121_plus.csv 메모 추가	보팅 개별 모델 가중치 [1,2,1]+ 피처 추가	2025-11-06 17:47:38	0.702
1346260	mp2_voting_rfetcat_weight121_2.csv 메모 추가	보팅 개별 모델 가중치[1,2,1]+ 하이퍼 파라미터 변경	2025-11-06 17:31:17	0.703
1346249	mp2_voting_rfetcat_weight121.csv 메모 추가	보팅 개별 모델 가중치[1,2,1]	2025-11-06 17:04:37	0.706

Method 3 모델링

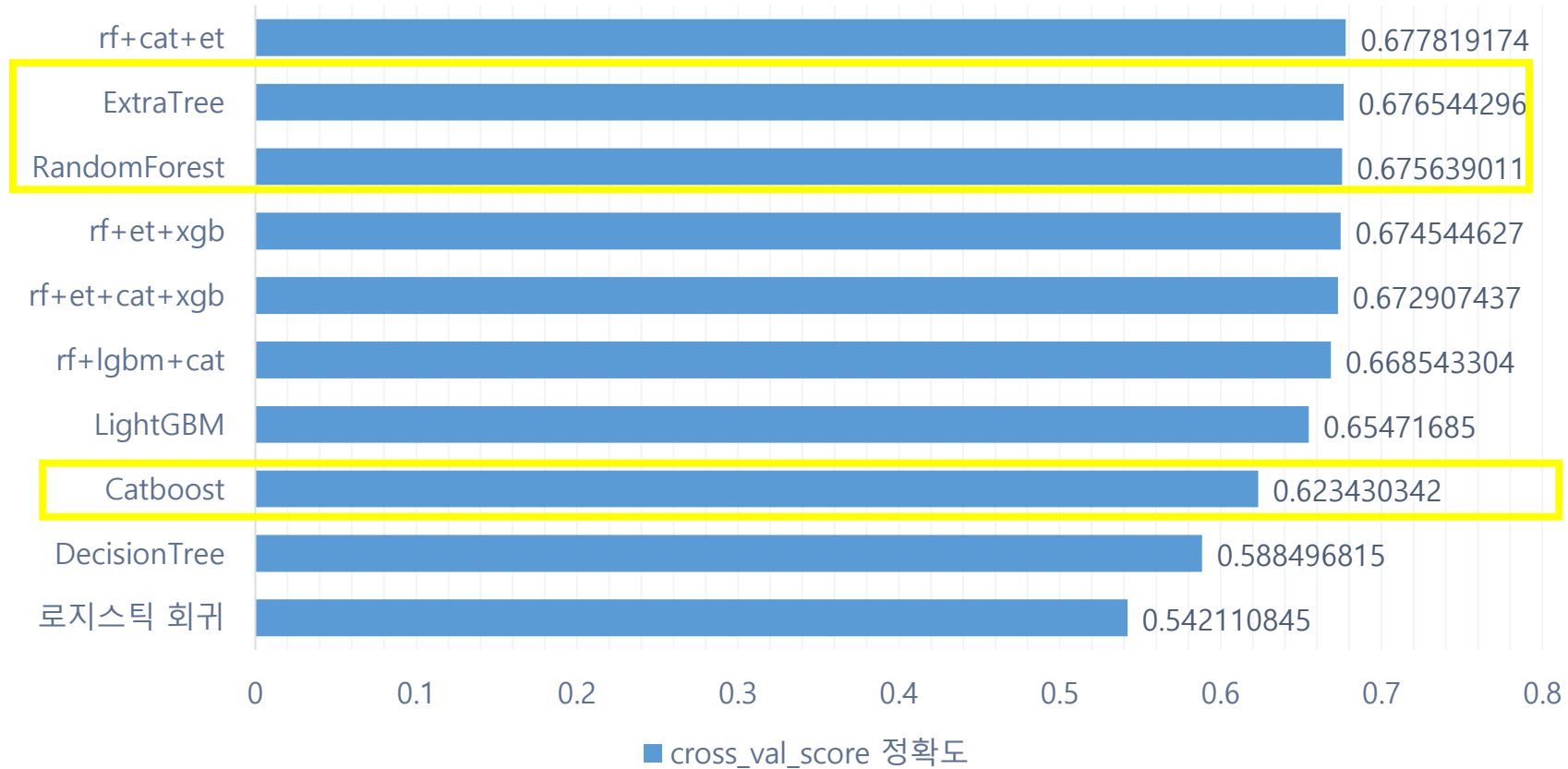
최적 모델 찾기

> 정확도 가장 높은 rf+et+cat 보팅 모델에서 가중치 변경

1346281	mp2_voting_rfetcat_weight122.csv 메모 추가	<u>보팅</u> 개별 모델 가중치[2,2,1] 2025-11-06 18:47	0.708
1346278	mp2_voting_rfetcat_weight131.csv 메모 추가	<u>보팅</u> 개별 모델 가중치[1,3,1] 2025-11-06 18:47	0.703

RF: ET: CAT=2:2:1

Method 3 모델링



1347236

mp2_voting_rfet.csv
메모 추가

RandomForest+ExtraTree 보팅

2025-11-09
15:49:25

0.7

Catboost 빼면
성능 하락

Method 3 모델링

최종 모델:

```
# ✅ 각 모델 정의
rf = RandomForestClassifier(
    n_estimators=500, max_depth=20, random_state=42
)

et = ExtraTreesClassifier(
    n_estimators=500, max_depth=25, random_state=42
)

cat = CatBoostClassifier(
    iterations=500, depth=8, learning_rate=0.05,
    loss_function='MultiClass', verbose=0, random_state=42
)

# ✅ 소프트 보팅
voting_clf = VotingClassifier(
    estimators=[('rf', rf), ('et', et), ('cat', cat)],
    voting='soft',
    weights=[2,2,1],
    n_jobs=-1
)

voting_clf.fit(train_x_mms, train_y)
y_pred=voting_clf.predict(test_mms)
submission=pd.read_csv('/content/drive/MyDrive/ESAA/mini_project/project2/sample_submission.csv')
submission['quality']=y_pred
submission.to_csv('mp2_voting_rfetcat_weight122.csv', index=False)
```

← 그리드 서치로 하이퍼파라미터 복잡하게
조정할수록 성능 하락 > 간단하게 지정

← 보팅 가중치 rf: et: cat=2:2:1
일 때 가장 높은 성능

← MinMaxScaling된 값으로 피팅

Method 3 모델링

최종 모델:

```
# ✅ 각 모델 정의
rf = RandomForestClassifier(
    n_estimators=500, max_depth=20, random_state=42
)

et = ExtraTreesClassifier(
    n_estimators=500, max_depth=25, random_state=42
)

cat = CatBoostClassifier(
    iterations=500, depth=8, learning_rate=0.05,
    loss_function='MultiClass', verbose=0, random_state=42
)

# ✅ 소프트 보팅
voting_clf = VotingClassifier(
    estimators=[('rf', rf), ('et', et), ('cat', cat)],
    voting='soft',
    weights=[2,2,1],
    n_jobs=-1
)

voting_clf.fit(train_x_mms, train_y)
y_pred=voting_clf.predict(test_mms)
submission=pd.read_csv('/content/drive/MyDrive/ESAA/mini_project/project2/sample_submission.csv')
submission['quality']=y_pred
submission.to_csv('mp2_voting_rfetcat_weight122.csv', index=False)
```

1346281	mp2_voting_rfetcat_weight122.csv 메모 추가	2025-11-06 18:30:14	0.708	
---------	---	------------------------	-------	---

제출 점수: 0.708